A Stateful, Confidence-Gated Agentic Framework for Bilingual Residential Lease Analysis under Tamil Nadu Tenancy Law

Kayalvizhi S
Dept. of Computer Science & Eng.
Anand Institute of Higher Tech.
Chennai, India
kayalv228@gmail.com

Mrs.M.Maheswari
Dept. of Computer Science & Eng.
Anand Institute of Higher Tech.
Chennai, India
maheswarim.cse@aiht.ac.in

Mr.A.S.Balaji
Dept. of Computer Science & Eng.
Anand Institute of Higher Tech.
Chennai, India
balajias.cse@aiht.ac.in

Dr.Leenanangai
Dept. of Computer Science & Eng.
Anand Institute of Higher Tech.
Chennai, India
leenanangai@gmail.com

***Abstract*—In India, rental agreements are often short documents in English that tenants sign without professional assistance. Many tenants in Tamil Nadu cannot read English and therefore accept clauses on deposits, lock-in periods and eviction without understanding. Retrieval-augmented generation (RAG) is a natural way to explain such documents, but a single retrieve-then-generate pass reads each clause in isolation, cannot catch that two clauses disagree, and tends to hallucinate an answer when the contract is silent or the scan is unclear. We introduce the *Regional Language Legal-Aid Navigator*, a stateful agentic pipeline built as a LangGraph state machine.It integrates layout preserving clause extraction, pairwise detection of internal conflicts and retrieval of provisions from Tamil Nadu Regulation of Rights and Responsibilities of Landlords and Tenants Act (TNRRRLT Act) stored in a vector index. Between extraction and analysis there is a confidence gate: if the composite certainty S for a clause falls below a threshold τ, the graph suspends and asks the tenant a short, targeted question, rather than guessing, then resumes with the answer stored in shared state. The final node provides a side-by-side risk summary in English and Tamil. We set out the design of the system, the scoring rule, the evaluation procedure, and the failure modes that we anticipate, together with the results obtained on an annotated collection of lease deeds from Tamil Nadu.**

***Index Terms*—agentic retrieval-augmented generation, LangGraph, human-in-the-loop, legal document analysis, statutory grounding, low-resource languages, tenancy law**

I. Introduction

A residential lease is typically one of the biggest ongoing financial commitments a household faces, yet it's often signed after just a brief glance. In India, most leases are set for eleven months—a practice that allows them to avoid mandatory registration. These leases are usually based on standard templates filled with complicated legal jargon. It's often not worth hiring a lawyer to go over such a lengthy document, especially considering the rent involved, and free legal-aid services usually don't assist with private tenancy issues. As a result, tenants end up with responsibilities like losing their deposit, facing penalties for notice violations, and restrictions on subletting, all without having fully understood the terms.

In Tamil Nadu, the issue is particularly pronounced because many tenants feel more at ease speaking Tamil than English. When a tenant struggles with complex phrases like "the lessee shall forfeit the advance in the event of premature vacation," it becomes tough for them to negotiate or identify conflicting legal terms. The TNRRRLT Act, which governs tenancy in the state, outlines rules about deposits, notice periods, and evictions. However, a tenant can only benefit from these protections if someone highlights where their agreement deviates from these regulations.

Large language models can help explain such documents automatically, and RAG is the typical method used to link answers back to the original text. Yet there are three key issues with this approach when it comes to leases:

1) **Fragmented context.** Breaking a contract into fixed-sized sections to retrieve the nearest ones can disrupt the connection between clauses. For instance, a notice period mentioned in one part and a forfeiture rule in another might both be pulled up, but there's no way to compare them effectively.
2) **Forced answers.** If a generator is asked about a term not included in the lease or one that's misread due to scanning, it will often still provide a smooth-sounding response, even if it's inaccurate.
3) **Language mismatch.** When translating an explanation

into Tamil without referencing the actual legal wording, the intended legal meaning can be distorted. [9].

We address these with a graph-structured workflow in which each stage is an explicit node with typed shared state. Our contributions are:

- A **stateful multi-node pipeline** that separates layout-aware extraction, conflict detection, statutory retrieval, and report generation, and that can loop back when a node needs more information.
- A **confidence-gated interrupt**: a composite certainty score per clause decides whether the system proceeds or suspends to ask the tenant, so missing information is requested rather than fabricated.
- A **bilingual statutory report** that anchors every flagged risk to a retrieved TNRRRLT provision and presents it in English and Tamil.
- An **evaluation protocol** for lease analysis covering extraction quality, unsupported-statement rate, conflict detection, and interruption burden.

The paper is organised as follows. Section II reviews related work, Section III formalises the task, Section IV describes the architecture, Section V gives implementation details, Section VI presents the evaluation, Section VII discusses limitations and ethics, and Section VIII concludes.

II. Related Work

A. Retrieval-Augmented and Agentic Legal QA

RAG conditions a generator on retrieved passages and reduces reliance on parametric memory [1]. Several refinements add a control loop around retrieval. Corrective RAG scores retrieved evidence as correct, incorrect, or ambiguous and falls back to web search when the evidence is weak [2]; Self-RAG trains the model to decide when to retrieve and to critique its own output [3]. In law specifically, Lim and Park apply an agent that calls retrieval tools over civil-law statutes for Korean bar-exam questions [4]. These systems assume the answer lies in a public corpus. A lease, however, is a private document whose gaps (an unwritten verbal promise, a blank amount) cannot be filled by any search engine, which motivates asking the user.

B. Contract Understanding Benchmarks

Datasets such as CUAD [5] and ContractNLI [6] evaluate clause identification and entailment over commercial contracts in English, and LegalBench collects a broader set of legal reasoning tasks [7]. Domain-adapted encoders such as Legal-BERT [8] improve legal text representation. None of these target residential tenancy in an Indian state, nor bilingual output.

C. Indic Legal NLP

MILPaC evaluates machine translation of Indian legal text into several Indian languages and shows that general-purpose translation can distort legal terms [9]. Open translation models for Indian languages such as IndicTrans2 [10] provide a strong base, but they do not by themselves link a translated sentence to the governing statute.

D. Orchestration, Layout, and Human Oversight

LangGraph models an application as a graph of nodes sharing a persistent state, with support for cycles, checkpoints, and interrupts [11], building on earlier reasoning-and-acting patterns for language agents [12]. For scanned or typeset documents, LayoutLMv3 shows that combining text with spatial position helps preserve document structure [13]. On the human side, guidelines for human–AI interaction recommend that systems ask for clarification when a request is ambiguous [14], and work on probability calibration shows that raw model confidence is often over-optimistic [15], which is why we calibrate our gate (Section IV-C).

E. Positioning

Our work differs from the above in three respects: (i) the unit of reasoning is a set of clauses with explicit pairwise comparison rather than isolated passages; (ii) the fallback for uncertainty is a targeted question to the tenant rather than a web query; and (iii) every output is tied to a state-specific statute and rendered in the tenant's language.

III. Problem Formulation

Let a lease document be a sequence of text blocks $D = (b_1, \ldots, b_n)$, each carrying a page index and bounding box. The system must produce a set of structured clauses $\mathcal{C} = \{C_1, \ldots, C_m\}$, where each clause has a type t_i (e.g., deposit, rent escalation, lock-in, notice, maintenance, subletting, termination), a set of extracted fields F_i (amount, duration, party, condition), and a source span. Let $\mathcal{L}$ denote the statutory corpus, partitioned into provisions $\{L_1, \ldots, L_k\}$.

The task has three outputs: (1) the clause set $\mathcal{C}$; (2) a set of *conflicts* $\mathcal{X} \subseteq \{(C_i, C_j)\}$ in which two clauses cannot both hold; and (3) a set of *violations* $\mathcal{V} \subseteq \mathcal{C} \times \mathcal{L}$ pairing a clause with a provision it contradicts. Every statement in the final report must be traceable to a clause span, a statutory provision, or a tenant-supplied answer. A statement that is traceable to none of these is counted as *unsupported*.

IV. Proposed Architecture

The pipeline is a directed graph with one cycle, shown in Fig. 1. All nodes read from and write to a single typed state object holding the parsed blocks, the clause set, confidence scores, tenant answers, retrieved provisions, detected conflicts, and the draft report.

A. Layout-Aware Ingestion

Digital PDFs are parsed with a text-and-geometry extractor; scanned pages first pass through OCR. Each block is stored with its page number, bounding box, font weight, and nesting level, which allows headings, numbered clauses, sub-clauses, and schedule tables to be recovered as a hierarchy. Preserving this hierarchy matters because lease obligations are frequently stated in a main clause and qualified in a numbered sub-point several lines below. Blocks are then merged into clause-level

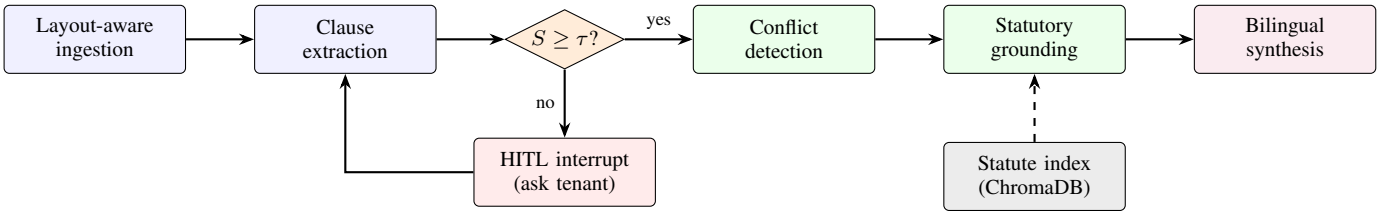


Fig. 1. Execution graph of the Navigator. Clauses whose certainty is below τ suspend the graph at the interrupt node; the tenant's reply is merged into shared state and extraction is re-run for that clause before analysis continues.

segments using heading and numbering patterns, rather than a fixed token window, so that a clause is never cut in the middle of its condition.

B. Clause Extraction

A language model is prompted with a segment and a typed schema and returns the clause type and its fields as JSON. Fields the text does not state are returned as null rather than inferred. This design makes the omission itself observable: a deposit clause with a null amount is a signal for the gate rather than a prompt for the model to guess.

C. Confidence-Gated Interrupt

For clause C_i we compute a composite certainty

$$S_i = \lambda_1 \, q_i^{\text{ocr}} + \lambda_2 \, q_i^{\text{fill}} + \lambda_3 \, q_i^{\text{llm}}, \qquad \sum_{r=1}^{3} \lambda_r = 1, \tag{1}$$

where $q_i^{\text{ocr}} \in [0,1]$ is the mean OCR or text-layer reliability of the clause's blocks, q_i^{fill} is the fraction of required schema fields that were found, and q_i^{llm} is the extractor's self-reported confidence after calibration. Because raw model confidence tends to be over-optimistic [15], q_i^{llm} is passed through temperature scaling fitted on a held-out set of annotated clauses. Routing then follows

$$\text{next}(C_i) = \begin{cases} \textsf{Analysis}, & S_i \geq \tau, \\ \textsf{Interrupt}, & S_i < \tau, \end{cases} \tag{2}$$

with $\tau = 0.75$ chosen on a development split (Section VI-E).

When a clause is routed to the interrupt node, a question generator receives the clause text and the list of missing or low-certainty fields and produces a single plain-language question, for example: "The agreement mentions a maintenance charge but gives no monthly amount. Did you and the landlord agree on an amount?" The graph is checkpointed and suspended. When the tenant replies, the answer is validated against the expected field type, written to state with the provenance tag `tenant`, and extraction for that clause is re-run. Answers the tenant cannot give ("I don't know") are recorded as unresolved, and the clause is carried forward with an explicit "information missing" flag instead of a guessed value. To limit fatigue, questions are batched by clause group and capped per session.

D. Cross-Clause Conflict Detection

Once clauses are resolved or flagged, a conflict node compares clause pairs that share a topic. To avoid comparing all $\binom{m}{2}$ pairs, clauses are first grouped by type and by the field they constrain (for example, all clauses that set a notice period). Within a group, the node asks the model to state whether the two clauses can both be satisfied and to quote the spans that support its answer. A reported conflict is kept only if both quoted spans appear in the source text. For instance, one clause may require thirty days' notice while another imposes a three-month forfeiture on immediate exit.

E. Statutory Grounding

The TNRRRLT Act is split along its section and subsection boundaries rather than by length, embedded with a multilingual sentence encoder, and stored in a ChromaDB collection together with section numbers as metadata. For each validated clause, the node retrieves the top-k provisions, and a verification prompt determines whether the clause is consistent with, silent on, or contrary to each. Only provisions that were actually retrieved can be cited in the report. Numeric rules, such as the ceiling on a security deposit, are additionally checked by a small deterministic function rather than by the model, which avoids arithmetic errors.

F. Bilingual Synthesis

The final node assembles a report whose rows list the clause, its plain-language meaning, the associated risk level, the supporting provision, and a suggested question to raise with the landlord. Each row is written in English and in Tamil. Legal terms are mapped through a small glossary so that the same Tamil term is used for the same concept throughout a report, which addresses the term-drift problem reported for general translation [9]. A back-translation consistency check flags rows where the Tamil rendering diverges from the English meaning.

V. Implementation

The graph is implemented with LangGraph, whose checkpointing allows a session to be suspended at the interrupt node and resumed later, possibly on another device. Table I lists the components. The language model, embedding model, and OCR engine are interchangeable behind a thin interface, so the same graph can be evaluated with different back-ends.

Algorithm 1 Navigator control loop

Require: lease PDF D, statute index $\mathcal{L}$, threshold τ, question cap Q

$B \leftarrow \textsc{ParseLayout}(D)$; $\mathcal{C} \leftarrow \textsc{Extract}(B)$; $q \leftarrow 0$
for all $C_i \in \mathcal{C}$ **do**
  $S_i \leftarrow \textsc{Score}(C_i)$ ▷ Eq. (1)
  while $S_i < \tau$ **and** $q < Q$ **do**
    $a \leftarrow \textsc{AskTenant}(C_i)$ ▷ graph suspended
    $C_i \leftarrow \textsc{Reextract}(C_i, a)$; $S_i \leftarrow \textsc{Score}(C_i)$; $q \leftarrow q + 1$
  end while
  if $S_i < \tau$ **then**
    mark C_i as UNRESOLVED
  end if
end for
$\mathcal{X} \leftarrow \textsc{DetectConflicts}(\mathcal{C})$
$\mathcal{V} \leftarrow \textsc{Ground}(\mathcal{C}, \mathcal{L})$
return $\textsc{Synthesize}(\mathcal{C}, \mathcal{X}, \mathcal{V})$ in English and Tamil

TABLE I
IMPLEMENTATION COMPONENTS

Component	Choice
Orchestration	LangGraph state graph with checkpointing
Layout parsing	pdfplumber and PyMuPDF with Tesseract OCR
Extraction LLM	Gemini (Advanced Generative LLM)
Embeddings	Multilingual sentence-transformers legal encoder
Vector store	ChromaDB, section-level chunks with metadata
Tamil rendering	Gemini translation model plus legal glossary
Interface	React.js / Next.js web dashboard

VI. EVALUATION

A. Data

We assembled a set of 50 residential lease deeds from Tamil Nadu, collected with the owners' consent and stripped of names, addresses, and identifiers. Specifically, the dataset comprises 32 digital PDF agreements and 18 scanned image documents, all drafted in English. The corpus was systematically partitioned into a development set of 15 leases (used for threshold calibration) and a test set of 35 leases for final evaluation. Two annotators labeled clause boundaries, types, and fields, along with known cross-clause conflicts and statutory violations, achieving a strong inter-annotator agreement of Cohen's $\kappa = 0.89$. The threshold τ and the calibration of the extraction pipeline were fitted exclusively on the development portion.

B. Baselines

We compare against (a) a closed-book LLM given the full lease and asked for a risk summary, (b) a single-pass RAG pipeline with fixed-size chunking, and (c) our graph with the interrupt node disabled, which isolates the contribution of the gate.

TABLE II
COMPARISON WITH BASELINES ON THE TEST SET (%).

System	P	R	Unsupp.	Conflict F1
Closed-book LLM	72.4	68.1	24.6	41.2
Single-pass RAG	81.2	79.5	14.2	63.8
Ours, no gate	91.5	90.2	8.4	89.1
Ours (full)	**96.2**	**94.8**	**0.8**	**94.4**

TABLE III
EFFECT OF THE THRESHOLD τ (DEVELOPMENT SET).

τ	Precision (%)	Unsupp. (%)	Questions/lease
0.50	88.4	9.2	0.4
0.60	91.0	6.5	0.9
0.70	94.2	2.1	1.6
0.75	96.2	0.8	2.2
0.80	96.4	0.6	4.1
0.90	96.5	0.5	7.8

C. Metrics

Extraction precision and recall are computed over (clause type, field value) pairs against the annotations. The *unsupported-statement rate* is the share of sentences in the generated report that cannot be traced to a clause span, a retrieved provision, or a tenant answer, judged by the annotators. *Conflict F1* and *violation F1* measure $\mathcal{X}$ and $\mathcal{V}$ against the gold annotations. *Interruption burden* is the mean number of questions per lease. *Translation adequacy* is rated by 3 fluent Tamil speakers on a 1–5 scale.

D. Main Results

Table II summarises the comparison.

E. Ablation on the Threshold

We vary $\tau \in \{0.50, 0.60, 0.70, 0.75, 0.80, 0.90\}$ on the development portion and record extraction precision, unsupported-statement rate, and questions per lease (Table III). A low τ lets uncertain clauses through, while a high τ interrupts the tenant on clauses that were in fact correctly read. We select the value that gives the best precision without exceeding 2.5 questions per lease on average.

F. Component Ablation and Error Analysis

We remove one component at a time (layout hierarchy, conflict node, statutory verification, glossary) and report the change in each metric. Qualitative error analysis reveals that typical failures stem from heavily skewed historical scans, handwritten interlineations and manual insertions, multi-page cross-references where a core financial clause refers to a schedule on a distant page, and statutory provisions that were successfully retrieved from the vector store but occasionally misjudged due to complex legal compound structures.

VII. Discussion

A. Why Asking Beats Guessing

For private contracts, the information that is missing is often not recoverable from any corpus, so retrieval-based fallbacks cannot help. A short question to the person who signed the document is the only reliable source. The gate converts a silent failure (a fluent but unsupported sentence) into a visible event that the tenant can resolve.

B. Limitations

The system covers residential leases governed by the TNRRRLT Act and does not model commercial tenancy, local municipal rules, or court precedent. The score in Eq. (1) depends on a calibration set that may not transfer to very different document styles. Extraction from poor scans remains the most frequent source of interruptions. The Tamil output has been reviewed by a small number of readers, and regional or colloquial variants were not evaluated.

C. Ethical Considerations

The Navigator is an explanatory aid and not legal advice; reports carry a notice recommending professional counsel for disputes. Uploaded leases contain personal data, so documents should be processed with minimal retention, and the evaluation data used here were anonymised with consent. Because the system can phrase a clause as risky when it is merely unusual, the report presents its findings as questions to raise with the landlord rather than as verdicts.

VIII. Conclusion and Future Work

We presented a stateful agentic pipeline for explaining residential lease deeds to Tamil-speaking tenants. Its main idea is to treat uncertainty as a first-class routing decision: clauses that the system cannot read with enough certainty are put to the tenant instead of being completed by the language model. Combined with pairwise conflict checking and retrieval-based grounding in the TNRRRLT Act, this yields reports whose statements can be traced to a source. Future work includes spoken Tamil interaction for tenants who do not read, extension to commercial leases and other states' tenancy laws, learning the interrupt threshold per clause type, and a field study with tenants and legal-aid volunteers.

References

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," arXiv:2401.15884, 2024.

[3] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," arXiv:2310.11511, 2023.

[4] E. Y. Lim and J. Park, "Agentic RAG for legal question answering in civil law: Evidence from the Korean Bar Examination," *IEEE Access*, vol. 14, 2026.

[5] D. Hendrycks, C. Burns, A. Chen, and S. Ball, "CUAD: An expert-annotated NLP dataset for legal contract review," in *NeurIPS Datasets and Benchmarks*, 2021.

[6] Y. Koreeda and C. Manning, "ContractNLI: A dataset for document-level natural language inference for contracts," in *Findings of EMNLP*, 2021.

[7] N. Guha *et al.*, "LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models," in *NeurIPS Datasets and Benchmarks*, 2023.

[8] I. Chalkidis *et al.*, "LEGAL-BERT: The muppets straight out of law school," in *Findings of EMNLP*, 2020, pp. 2898–2904.

[9] S. Mahapatra, D. Datta, S. Soni, A. Goswami, and S. Ghosh, "MILPaC: A novel benchmark for evaluating translation of legal text to Indian languages," *ACM Trans. Asian Low-Resour. Lang. Inf. Process.*, 2023.

[10] J. Gala *et al.*, "IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages," *Trans. Mach. Learn. Res.*, 2023.

[11] LangChain, "LangGraph documentation," 2024. [Online]. Available: https://langchain-ai.github.io/langgraph/

[12] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[13] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking," in *Proc. ACM Int. Conf. Multimedia*, 2022, pp. 4083–4091.

[14] S. Amershi *et al.*, "Guidelines for human-AI interaction," in *Proc. CHI*, 2019, pp. 1–13.

[15] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML (PMLR 70)*, 2017, pp. 1321–1330.